ChunavIQ: An Open, End-to-End Verifiable Remote-Voting Prototype for India's Legal and Identity Constraints, with Deniable Revoting, Hardware-Held Trustee Keys and Adversarial Dress Rehearsals

Ashish Ramtekkar[0009−0006−5516−6843]

Independent researcher, Mumbai, India
itsashiish@proton.me

Abstract. India runs the world's largest elections: about 97 crore (970 million) electors in 543 parliamentary constituencies in 2024. Any remote-voting proposal for India must fit its election law, its identity infrastructure and its scale, and must say what it does not solve. We present ChunavIQ, an open-source research prototype that combines established techniques—threshold ElGamal, zero-knowledge ballot proofs, deniable revoting, Terelius–Wikström verifiable shuffles, designated-verifier proofs and an append-only Merkle log—into a complete system adapted to Indian constraints. Voters register a phone with either Aadhaar or PAN; because PAN verification only matches a name and birth date, the PAN path adds a one-time code to the mobile number on the electoral roll and a face match. A voter may vote again at any time, and a vote at a polling station on polling day cancels the online ballot; nobody, including a coercer, can tell which ballot counted. One national key is shared among trustees whose shares stay inside hardware security modules. We contribute (i) the adapted protocol and two independent implementations with two independent verifiers that cross-check each other's records; (ii) a capacity model for 100–150 crore electors built from measured unit costs; (iii) an open dress-rehearsal method that runs a constituency election over HTTP under seven kinds of attack and publishes every request; and (iv) a provision-by-provision analysis of the Indian legal changes such a system would need. Every record we produced verified. A first rehearsal forced the largest constituency's full design rate onto one machine and recorded only 72–99% of honest ballots per phase; after fixes identified from its published metrics, a rehearsal at 78% of that rate on the same machine recorded every honest ballot, 97–100% of them within 30 s, while refusing every attack. ChunavIQ uses synthetic data only and is not proposed for binding elections.

Keywords: Electronic voting · End-to-end verifiability · Coercion resistance · Revoting · Verifiable shuffle · Hardware security modules · India

1 Introduction

Indian parliamentary elections are conducted by the Election Commission of India (ECI) with electronic voting machines (EVMs) and voter-verified paper audit trails (VVPAT) at polling stations. The 2024 general election had about 96.8 crore electors; the largest constituency (Malkajgiri) had about 37.8 lakh (3.78 million). Remote voting is periodically proposed for migrants, the elderly and others who find polling stations hard to reach. The research literature is clear that internet voting raises problems that polling stations solve physically—coercion, malicious devices, denial of service, and the need for public evidence that the result is right [21,5,28,15].

This paper does not propose a new cryptographic primitive, and it does not argue that India should adopt internet voting. It asks a narrower, engineering question: *if* remote voting were considered for India, what would a design look like that (a) produces public, independently checkable evidence of the result, (b) fits Indian identity infrastructure and election law, (c) bounds coercion in the way India's polling-station system allows, and (d) can be costed and tested at national scale? We answer with a working, open prototype and measured evidence.

Contributions.

1. **An adapted protocol** (Sect. 4) combining deniable revoting with a polling-day override that uses India's existing polling stations; Aadhaar-or-PAN registration in which the weaker PAN path is strengthened with factors held by the election authority; one national threshold key whose shares stay inside hardware security modules (HSMs) under operator and observer quorums; and a federation of 543 independent constituency shards.
2. **Two implementations and two verifiers** (Sect. 6): a Python engine and a TypeScript browser engine that write byte-identical records, and two independently written verifiers, each of which checks records produced by the other implementation.
3. **A national-scale cost model** (Sect. 7.1) built from measured unit costs, giving cores, bandwidth, storage and post-close computation for 100 and 150 crore electors.
4. **An open dress-rehearsal method** (Sect. 7.2) that runs a full constituency election over HTTP under seven concurrent attack kinds and publishes every request and every second of server state, including a run that exposed availability defects and the rerun after fixing them.
5. **A legal gap analysis** (Sect. 8) listing, provision by provision, what Indian law would need to say.

All data are synthetic. No real electoral roll, Aadhaar, PAN, phone number or photograph is used, and no real election infrastructure was accessed or tested. Source code, records and rehearsal outputs are released under the Apache-2.0 licence (see Data Availability).

2 Background and Related Work

Verifiable voting. Homomorphic tallying of exponential ElGamal [12] ciphertexts with threshold decryption [22,13] and disjunctive 0/1 proofs [9,10] underlies Helios [2] and Belenios [8]. Benaloh's cast-or-challenge [4] lets voters audit their device. Bernhard et al. [5] survey what public evidence an election should produce. Security reviews of deployed systems in Estonia [28] and Switzerland [15] show how implementation and proof errors undermine verifiability; we draw two lessons from them: verify everything from a published record, and verify it with more than one implementation.

Coercion resistance. Juels, Catalano and Jakobsson (JCJ) [17] and Civitas [7] give voters fake credentials indistinguishable from real ones; this is strong but asks voters to manage secret credentials and costs heavy tallying. *Revoting*—only the last ballot counts—is used in Estonia, where a paper vote also overrides an internet vote [28]. Achenbach et al. [1] formalise deniable revoting, and VoteAgain [20] makes it scale, using dummy ballots so that even the tally server does not learn how often or when a voter revoted.

How ChunavIQ differs. Our coercion mechanism is a simpler, *weaker* variant of deniable revoting: the ballot box learns which ballots share a voter's tag (but not whose tag it is), so a collusion of the ballot box and the eligibility authority learns *who voted when*, which VoteAgain avoids. In exchange the finalization step is a per-ballot OR-proof followed by a standard shuffle, and counted tags are checked for distinctness against a published list of valid registrations. What we add is not cryptographic novelty but a full system tied to Indian constraints: the polling-station override, Aadhaar-or-PAN identity, HSM custody of one national key, a 543-shard federation, measured national costs, open adversarial rehearsals, and a legal gap analysis.

3 Requirements and Threat Model

Requirements from Indian law and practice. A person may vote only once and in one constituency (s.62, Representation of the People Act 1951 [25]); officials must keep votes secret (s.128); ties are decided by lot by the Returning Officer (s.65), so software must report, not break, ties; NOTA must appear on every ballot and is counted but cannot win [23]; Aadhaar linkage to the electoral roll is voluntary, so Aadhaar cannot be the only route; any State use of personal data must be lawful, necessary and proportionate [24,11]; and candidates must be able to challenge results, as the Supreme Court's VVPAT judgment [3] reaffirmed.

Roles. An *eligibility authority* (roll and identity), a *ballot box* (sequencer and public board), an *election officer* (lifecycle and publication), n *trustees* with HSMs (threshold k), independent *observers*, and polling stations. Roles are meant to be held by different institutions.

Assumptions. (A1) At most $k-1$ trustees collude and at least one mixer per batch is honest. (A2) The eligibility authority and ballot box do not share logs. (A3) Observers obtain the manifest hash out of band. (A4) Discrete logarithm and DDH are hard in the 256-bit-order subgroup of a 2048-bit prime field; SHA-256 behaves as a random oracle for Fiat–Shamir [14]. (A5) HSMs enforce their applet's policy. (A6) A coerced voter has some private moment during the voting week or on polling day.

Out of scope. A coercer controlling a voter for the entire week and polling day; a malicious device that also evades Benaloh challenges; network-layer floods (assumed absorbed by a CDN or anycast scrubbing provider); anonymity against a network-wide observer.

4 Protocol

4.1 Setup

The group $\mathbb{G}_q$ has a 2048-bit prime p and 256-bit prime $q \mid p-1$, derived deterministically from a public seed with SHAKE-256 and Miller–Rabin tests, so anyone can re-derive it. Every value an adversary supplies that is later raised to a challenge is checked for subgroup membership. Fiat–Shamir challenges hash a domain-separated, injective type-length-value encoding. The officer publishes a signed *manifest* (constituencies, options ending in NOTA, voting window, trustee keys, mixing and revoting parameters, software digest); its SHA-256 hash is what observers pin.

4.2 Key ceremony inside HSMs

One national ceremony produces the key $K = g^s$ used by every constituency, so each trustee holds *one* share, not 543; per-constituency keys would multiply secrets to guard without adding protection, since the same trustees would hold them all. Each trustee's HSM runs an applet that performs Feldman VSS [13,22]: it commits to a degree-$(k-1)$ polynomial with Schnorr proofs [26], sends shares to other HSMs encrypted under their X25519 transport keys (HKDF-SHA256, AES-256-GCM), checks incoming shares, and signs an acknowledgement with an attestation key. Standard PKCS#11 mechanisms cannot compute a Chaum–Pedersen decryption share without exporting the share, which is why custom code inside the HSM boundary (as offered by several vendors' SDKs) is required. The applet produces decryption shares only when a request carries the officer-signed CLOSED lifecycle entry, approvals from 2 of 3 operator cards, attestations from 2 of 3 observers that the ciphertexts are what the stated purpose says (totals, or outputs of verified shuffles), a fresh nonce, and remaining decryption budget. Every operation is appended to a hash-chained, signed log that is published after the election.

4.3 Registration: Aadhaar or PAN

A voter proves they are a particular entry on the electoral roll by one of: (i) Aadhaar and a one-time password (OTP) to the Aadhaar-registered mobile; (ii) Aadhaar and face authentication; or (iii) PAN, *plus* an OTP to the mobile number on the electoral roll, *plus* a face match against the roll photograph. Path (iii) exists because Aadhaar linkage is voluntary, and its extra factors exist because PAN verification only confirms that a PAN, name and birth date match—details printed on the card and widely shared. All three resolve to one roll entry, so holding both documents does not give two registrations. Identifiers are stored as keyed hashes; five failures lock the entry.

The voter's phone draws $t \in \mathbb{Z}_q$ and registers the *tag* $T = g^t$ with a Schnorr proof bound to the authenticated session. Re-registration from a new phone (at most five times) replaces the tag and silently revokes the old one. A voter who votes at a polling station on polling day has their online tag revoked. Revocations are *not* published while voting is open, and the ballot box keeps accepting ballots under revoked tags, so a coercer holding an old phone cannot detect the revocation.

4.4 Ballot

For options $o_1, \ldots, o_m$ (the last is NOTA) and choice o^*:

- tag ciphertext $(\alpha_T, \beta_T) = (g^\rho, TK^\rho)$;
- per option, $(\alpha_o, \beta_o) = (g^{r_o}, g^{[o=o^*]}K^{r_o})$ with a disjunctive 0/1 proof [9] whose context includes α_T;
- a Chaum–Pedersen proof [6] that the product of the option ciphertexts encrypts g^1;
- a proof of knowledge of (ρ, t) for the tag ciphertext, bound to a digest of the selections and limit proof, so the encrypted tag cannot be transplanted onto another vote.

The tracking code is the SHA-256 of the canonical ballot. Before casting, a voter may spoil a ballot (Benaloh challenge [4]): the device reveals its randomness, anyone re-encrypts and compares, and a fresh ballot is prepared.

4.5 Submission and acceptance

The phone sends the ballot together with the plaintext T and a *link proof* that $(\alpha_T, \beta_T/T)$ encrypts zero. T and the link proof go only to the ballot box and are never published; the public record holds only the encrypted tag. The ballot box checks the schema exactly (no free fields that could mark a ballot), subgroup membership, all proofs, that T is registered, the voting window, at most 30 ballots per tag, and ciphertext uniqueness. An identical retry under the same tag returns the original receipt; the same ballot under another tag is refused as a copy. Accepted ballots are appended to an RFC 6962 Merkle log [18,19] with signed, consistency-proven checkpoints, and the voter receives an inclusion proof.

4.6 Finalization: marking the last ballot, in secret

After close, the eligibility authority publishes a signed registration summary (eligible, registered, voted in person, re-registrations) and two sorted lists: salted participation commitments (each voter can find their own) and *valid tags* (registered and not overridden in person). The ballot box then sets, for every ballot in record order, $f = 1$ if and only if it is the last ballot under its tag and that tag is valid, and publishes

$$F = \mathsf{Enc}(g^f; u), \qquad M_o = \begin{cases} (\alpha_o g^{w_o}, \beta_o K^{w_o}) & f = 1 \\ (g^{w_o}, K^{w_o}) & f = 0 \end{cases}$$

with an OR-proof that either [F encrypts 0 and every M_o encrypts 0] or [F encrypts 1 and every M_o re-encrypts the ballot's option]. The product of the M_o over all ballots encrypts exactly the counted votes.

4.7 Mixing and the counted-tag check

Rows $(\alpha_T, \beta_T; F)$ are split into batches and each batch is re-encrypted and permuted by at least three different trustees, each publishing a Terelius–Wikström proof of shuffle [29] using generators derived from a public seed. Only on the final output do k trustees publish proven decryption shares of the flags, and then of the tags of rows with $f = 1$. Verifiers check that the decrypted counted tags are *distinct* and that each is in the published valid-tag list. Because of the shuffle, nobody—including the ballot box—can link a counted tag back to a ballot.

4.8 Tally

The officer publishes the per-option product of masked votes; k trustees publish Chaum–Pedersen-proven decryption shares; the count is recovered from g^c by baby-step giant-step [27] (36 ms for a range of 60 lakh). The number of counted ballots must equal the number of counted tags. The winner is first-past-the-post among candidates; NOTA cannot win; ties are reported for the Returning Officer.

4.9 What the voter can check

A voter checks inclusion of every ballot they cast. To learn that their *last* ballot counted, they ask the ballot box for a designated-verifier proof [16]: an OR-proof of [flag of this ballot encrypts 1] or [I know t with $T = g^t$]. The voter knows t, so they could have produced the proof themselves; it convinces them and nobody else. Their phone can produce an equally valid-looking proof for *any* ballot, including one cast under a coercer's eyes.

4.10 National federation

Each constituency is an independent shard with its own sequencer, board and eligibility service; tags and all proofs are bound to one constituency, so the voting path has no cross-shard coordination. A signed national manifest fixes the shard set; a signed national ceremony file is copied into every shard record; a signed shard index binds each shard's manifest hash and final checkpoint. The federation verifier requires every declared shard to verify independently, to use the national key, and to sum to the published national result.

5 Security Discussion

We give informal arguments; we have *not* produced formal proofs or machine-checked models, and we consider that the most important next step.

Eligibility and one counted vote. A ballot needs a registered tag and proof of its secret; finalization OR-proofs bind flags to ballots; counted tags are shown distinct and valid after mixing. A ballot box that flags two ballots of one voter, or a ballot under a revoked tag, yields a repeated or invalid counted tag and fails verification; both attacks are among our tamper tests.

Integrity of the count. Every transformation from ballots to totals (finalization, each shuffle, every decryption) carries a zero-knowledge proof checked from the public record by two independent verifiers.

Secrecy. Votes are decrypted only as totals; flags and tags only after shuffling by at least three trustees. Linking a vote to a person requires $\geq k$ trustees, every mixer of the relevant batch, the ballot box and the eligibility authority.

Coercion (bounded). A coercer who watches a vote, or holds a receipt, cannot tell whether that ballot was later superseded, because the record does not reveal which ballots share a tag or which ballot counted, and the voter's "it counted" proof is fakeable. Residual leaks: (i) a coercer who copied t can see from the published valid-tag list after close that the voter re-registered or voted in person (not that they merely revoted); (ii) the ballot box together with the eligibility authority learns who voted when.

6 Implementation

The reference engine is Python (with optional GMP). A second engine in TypeScript runs a complete election in a browser tab, using Web Workers for parallel cryptography, as a public demonstration. The two engines produce byte-identical canonical JSON records. Two verifiers were written separately from the engines and from each other: a Python package that imports nothing from the election code, and a TypeScript verifier. Continuous integration runs the Python verifier on browser-made records and the TypeScript verifier on Python-made records.

The record format is an append-only JSON-lines file with Ed25519-signed bodies and signed tree heads. The ballot-intake service is an asyncio HTTP server with defences ordered cheapest first: connection caps and header/body

Table 1. Measured unit costs (one core; 16-option ballot unless stated).

Operation	Cost
Voter device: encrypt and prove ballot, tag proof and link	37 ms
Server: verify ballot and tag link	59 ms
Ballot box: finalize one ballot (flag, masked votes, OR-proof)	40 ms
Verify one finalization	51 ms
One mixer: shuffle and prove, per row	3.5 ms
Verify one shuffle, per row	5.0 ms
Trustee decryption share with proof / verify	1.0 / 1.8 ms
Verify a registration proof	0.8 ms
Recover a count up to 60 lakh (baby-step giant-step)	36 ms
Record size: ballot / its finalization / shuffle row	56.6 / 19.8 / 3.2 KiB
Sequencer ingest, one shard, group commit with `fsync`	2,614 ballots/s

deadlines, size limits, per-client token buckets, load shedding with `Retry-After`, exact schema checks, idempotent replay without proof work, the registered-tag check, a per-tag minimum interval (so a stolen registration buys at most one expensive check per interval), proof checks in a bounded process pool, and group commit (one transaction and one `fsync` per batch, receipts only after `fsync`).

Testing. The test suite maps 19 security invariants to tests. The main suite has 202 tests (functional rules, identity paths, concurrency races, crash recovery, privacy of records and logs, property-based fuzzing of ballots, and insider/outsider tampering, including a ballot box that counts a revoked tag or two ballots of one voter, and a mixer that replaces a row). A further 17 tests cover national-scale properties (counting at 58 lakh, federation, bounded verifier memory, ingest rate), one end-to-end election with 2,000 voters checks the result against ground truth, and the browser suite drives the full user flow including eight refused attacks and five detected tampers. Mutation checks confirmed that a regression test fails on a defect we found and fixed (an idempotent-retry path that, under a race, could return an original receipt to a copied ballot under a different tag).

7 Evaluation

All measurements are on one 16-core desktop (Linux, Python 3.14, GMP), 2048-bit group.

7.1 Unit costs and national model

Table 1 lists measured costs. The national model (Table 2) assumes a seven-day window with 16 voting hours a day, a busiest hour twice the mean, a last-day surge of 2×, 2× headroom, 10% extra ballots from revotes, 16 options typical and 50 in the worst seat. Two observations matter. First, intake is a CPU-provisioning problem, not a request-rate problem: a few thousand cores verify

Table 2. Modelled national election from measured unit costs.

	100 crore, 65%	150 crore, 100%
Ballots cast incl. revotes	71.5 crore	165 crore
Design rate (busiest hour × headroom)	14,187/s	32,738/s
Verifier cores at design rate (16 / 50 options)	838 / 2,407	1,935 / 5,555
Ingress at design rate	6.6 Gbit/s	15 Gbit/s
Largest seat: design rate / ingest headroom	55/s / 47×	128/s / 20×
Public record (ballots, finalization, shuffles, shares)	70 TB	161 TB
After close: core-hours / wall time on 2,000 cores	14,063 / 7.0 h	32,453 / 16 h
Full independent re-verification on 2,000 cores	13 h	31 h
Cores for observers verifying continuously	105	242

ballots at the design rate, and constituency shards share nothing, so capacity adds linearly. Second, the after-close work introduced by revoting (finalization, three shuffles, attestation, decryption) is the expensive part: about 32,000 core-hours at 150 crore electors and 100% turnout, or 16 h on 2,000 cores. Finalization proofs can be precomputed during the week and redone only for superseded ballots, so this is an upper bound for that part.

7.2 Dress rehearsals under attack

A rehearsal runs one constituency election end to end with a real clock: HSM key ceremony, identity checks and registration, pre-encryption of ballots and about 10% revotes on simulated devices, the hardened server as a separate process, Poisson honest arrivals in three phases—baseline, under attack, and a surge at 1.5×—while seven attack kinds run concurrently (junk bodies, oversized bodies, unregistered tags, broken proofs from registered attackers, replays, slowloris and idle-connection floods), followed by finalization, mixing, HSM-gated decryption and independent verification. Every request and every second of server state are written to disk and published.

Table 3 shows three runs. The *smoke* profile recorded every honest ballot at sub-second latency while refusing all attacks: junk was rejected as malformed, oversized bodies at the header, unregistered tags by a database lookup, broken proofs after at most one expensive check per attacker tag per interval, and slow or idle connections were closed by the server.

In run 1, the *mini-peak* profile ran the largest constituency's full design rate (128 ballots/s, 16 options) for two minutes on the same single machine that also generated the honest load and the attacks. Integrity held: the record verified, and every receipt issued was on the board. Availability did not: 72–99% of honest ballots were recorded per phase, with p95 latency up to 30 s. The unrecorded ballots were never accepted; their simulated phones stopped after ten attempts. From the published server metrics we identified four causes: (i) the server's proof workers, the load generator and the attackers competed for the same 16 cores; (ii) parsing and duplicate lookups ran on the network thread; (iii) the

Table 3. Dress rehearsals on one 16-core machine. All records VERIFIED; every receipt issued to an honest voter was on the board; no attack request was accepted. For run 2, the parenthesised figure is the share recorded within 30 s of the first attempt.

Run	Phase	Honest	Recorded	p95	Attacks
smoke	baseline	50	100%	0.09 s	0
(4 options, 2/s)	under attack	84	100%	0.09 s	1,210
	surge 1.5×	73	100%	0.09 s	469
mini-peak, run 1	baseline	4,514	99.0%	7.6 s	0
(16 options, 128/s)	under attack	6,149	83.4%	21.7 s	8,426
	surge 1.5×	6,840	72.3%	29.7 s	3,097
mini-peak, run 2	baseline	3,606	100% (100%)	0.65 s	0
(after fixes,	under attack	4,728	100% (100%)	0.60 s	8,192
99.7/s)	surge 1.5×	5,345	100% (97.3%)	26.8 s	3,139

process hit the default limit of 1,024 open files, turning 228 requests into internal errors rather than retryable refusals; and (iv) simulated phones gave up after ten attempts, whereas real phones would keep trying across a seven-day window. We changed the server to raise its file limit and cap connections below it, to answer any internal failure with a retryable `503` (the batch has rolled back, so nothing was recorded), and to move parsing and lookups off the network thread; we changed the rehearsal to report both "recorded in the end" and "recorded within 30 s", to let phones retry until 60 s after the phase ends, and to size the honest rate to what the machine sustains while leaving three cores for the load generator.

Run 2, on the same machine, sustained 99.7 honest ballots/s (77.9% of the design rate; 150 ballots/s during the surge) while the same attacks ran. Every honest ballot was recorded in every phase; all were recorded within 30 s in the baseline and attack phases (p95 0.6 s), and 97.3% in the surge, where p95 latency rose to 27 s and phones needed 3.8 attempts on average. No internal errors occurred; the server ran 13,947 expensive proof checks for 13,679 honest ballots, so attackers forced only a few hundred. Full design rate on one machine that also generates the load remains out of reach; in the intended deployment, proof checking runs on a separate, horizontally scaled fleet. We report the failed run deliberately: an open rehearsal is useful precisely because it can fail in public.

8 Legal and Deployment Considerations in India

Software readiness is not legal authority. Remote online voting is not a prescribed manner of voting for parliamentary or assembly elections. Table 4 summarises changes a lawful deployment would need; it is an orientation for discussion, not legal advice.

Table 4. Indicative legal changes (not legal advice).

Instrument	Change needed
RP Act 1951, s.59, s.60, s.61A	Prescribe certified remote electronic voting as a manner of voting, for notified classes or all electors, with rules for keys, mixing, counting and publication.
RP Act 1951, s.62	Define the counted vote when revoting is allowed: the last online ballot under a valid registration, cancelled by an in-person vote.
RP Act 1951, s.128	Extend secrecy duties to ballot-box operators, identity services, trustees, mixers and HSM operators, including a duty not to combine what they know.
Election petitions; s.65	Treat independent re-verification of the public record as the equivalent of a recount; keep tie-breaking a witnessed human act.
Conduct of Election Rules 1961	A new part for registration, revoting, override, key ceremony, observers, finalization, mixing, publication and retention.
Aadhaar Act 2016 and authentication rules	Express authorisation for electoral authentication meeting the *Puttaswamy* proportionality test [24].
PAN (Income-tax law)	A legal basis and data-sharing arrangement to verify PAN for elector registration only.
DPDP Act 2023 [11]	Compliance or a narrow notified exemption; retention limits on identity linkage.
IT Act 2000 (s.70)	Protected-system status, independent security testing, incident reporting, responsible disclosure.
Bharatiya Nyaya Sanhita 2023	Offences for demanding to watch an online vote and for buying, selling or seizing voting phones or registrations.
Certification and observers	Public criteria, audits, published source and verifier, and a statutory right for candidates and observers to obtain and verify the record.

9 Limitations

- **No formal security proofs** and no independent cryptographic review; the author also wrote both verifiers, which reduces but does not remove the risk of a shared misunderstanding of the specification.
- **Coercion resistance is bounded**, with the residual leaks of Sect. 5; it fails against a coercer who controls a voter for the whole period.
- **Registration of abstainers**: a dishonest eligibility authority can register tags for voters who never vote; publication of counts and commitments makes this auditable, not impossible. Splitting registration across two authorities is future work.
- **Malicious devices** can leak votes or hide the challenge interface; Benaloh challenges reduce but do not remove this.
- **Availability**: on one machine that also generated the load, every honest ballot was recorded at 78% of the largest constituency's design rate, but not

at 100% (Sect. 7.2); a multi-machine national rehearsal, network-layer DDoS testing with a provider and failover drills remain undone.
- **HSM custody** is emulated in software; the applet has not been ported to or evaluated on vendor hardware.
- **Performance**: an elliptic-curve group would reduce record size and CPU several-fold; we have not measured it.
- **Synthetic identity**: Aadhaar, PAN, SMS and face matching are simulated; SIM-swap and presentation attacks are not evaluated.

10 Conclusion

ChunavIQ shows that established verifiable-voting techniques can be assembled into a complete, testable system shaped by Indian law, identity infrastructure and scale, with public evidence at every step: two verifiers, a national cost model from measured costs, and rehearsals that publish their failures as well as their successes. It also shows the limits: coercion is bounded rather than solved, the after-close computation added by revoting is substantial, a single machine that also generates the load sustains about four-fifths of the largest constituency's design rate under attack, and the law would have to change first. We hope the prototype is useful as a reference point for discussion, review and teaching, not as a deployment candidate.

Ethics Statement. All voters, identities, candidates, parties and results are synthetic and generated in formats that real systems do not issue; a scanner in continuous integration rejects any value shaped like a real Aadhaar, PAN, EPIC or Indian mobile number. No real electoral roll, government service or election infrastructure was accessed, imitated or tested. Every record carries a notice that it has no legal effect.

Author Contributions and Use of AI Tools. The author conceived the project, defined its goals and requirements for the Indian context, made the design decisions (deniable revoting with a polling-day override, Aadhaar-or-PAN registration, a single national key held in hardware security modules, and the open dress-rehearsal method), directed and reviewed all work, and ran every experiment reported here. An AI assistant (Anthropic's Claude) was used as a tool for software implementation, test construction, documentation and drafting of this manuscript under the author's direction. The author takes full responsibility for the content. All numerical results come from runs of the released code.

Data Availability. Source code (Apache-2.0), specifications, test suite, election records, rehearsal outputs (every request and per-second server metrics) and the capacity model are available at `https://github.com/itsashiish/chunaviq`. An interactive demonstration runs at `https://chunaviq.oglic.com`.

References

1. Achenbach, D., Kempka, C., Löwe, B., Müller-Quade, J.: Improved coercion-resistant electronic elections through deniable re-voting. USENIX Journal of Election Technology and Systems (JETS) **3**(2), 26–45 (2015)
2. Adida, B.: Helios: Web-based open-audit voting. In: Proceedings of the 17th USENIX Security Symposium. pp. 335–348. USENIX Association (2008)
3. Association for democratic reforms v. election commission of india. Supreme Court of India, Writ Petition (Civil) No. 434 of 2023, judgment of 26 April 2024 (2024)
4. Benaloh, J.: Ballot casting assurance via voter-initiated poll station auditing. In: Proceedings of the USENIX Workshop on Accurate Electronic Voting Technology (EVT '07). USENIX Association (2007)
5. Bernhard, M., Benaloh, J., Halderman, J.A., Rivest, R.L., Ryan, P.Y.A., Stark, P.B., Teague, V., Vora, P.L., Wallach, D.S.: Public evidence from secret ballots. In: Electronic Voting – E-Vote-ID 2017. LNCS, vol. 10615, pp. 84–109. Springer (2017)
6. Chaum, D., Pedersen, T.P.: Wallet databases with observers. In: Advances in Cryptology – CRYPTO '92. LNCS, vol. 740, pp. 89–105. Springer (1992)
7. Clarkson, M.R., Chong, S., Myers, A.C.: Civitas: Toward a secure voting system. In: 2008 IEEE Symposium on Security and Privacy. pp. 354–368. IEEE (2008)
8. Cortier, V., Gaudry, P., Glondu, S.: Belenios: A simple private and verifiable electronic voting system. In: Foundations of Security, Protocols, and Equational Reasoning, LNCS, vol. 11565, pp. 214–238. Springer (2019)
9. Cramer, R., Damgård, I., Schoenmakers, B.: Proofs of partial knowledge and simplified design of witness hiding protocols. In: Advances in Cryptology – CRYPTO '94. LNCS, vol. 839, pp. 174–187. Springer (1994)
10. Cramer, R., Gennaro, R., Schoenmakers, B.: A secure and optimally efficient multi-authority election scheme. In: Advances in Cryptology – EUROCRYPT '97. LNCS, vol. 1233, pp. 103–118. Springer (1997)
11. The digital personal data protection act, 2023 (act no. 22 of 2023). Government of India (2023)
12. ElGamal, T.: A public key cryptosystem and a signature scheme based on discrete logarithms. IEEE Transactions on Information Theory **31**(4), 469–472 (1985)
13. Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: 28th Annual Symposium on Foundations of Computer Science (FOCS). pp. 427–437. IEEE (1987)
14. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Advances in Cryptology – CRYPTO '86. LNCS, vol. 263, pp. 186–194. Springer (1987)
15. Haines, T., Lewis, S.J., Pereira, O., Teague, V.: How not to prove your election outcome. In: 2020 IEEE Symposium on Security and Privacy. pp. 644–660. IEEE (2020)
16. Jakobsson, M., Sako, K., Impagliazzo, R.: Designated verifier proofs and their applications. In: Advances in Cryptology – EUROCRYPT '96. LNCS, vol. 1070, pp. 143–154. Springer (1996)
17. Juels, A., Catalano, D., Jakobsson, M.: Coercion-resistant electronic elections. In: Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society (WPES). pp. 61–70. ACM (2005)
18. Laurie, B., Langley, A., Kasper, E.: Certificate transparency. RFC 6962, Internet Engineering Task Force (2013)

19. Laurie, B., Messeri, E., Stradling, R.: Certificate transparency version 2.0. RFC 9162, Internet Engineering Task Force (2021)
20. Lueks, W., Querejeta-Azurmendi, I., Troncoso, C.: VoteAgain: A scalable coercion-resistant voting system. In: Proceedings of the 29th USENIX Security Symposium. pp. 1553–1570. USENIX Association (2020)
21. Park, S., Specter, M., Narula, N., Rivest, R.L.: Going from bad to worse: From Internet voting to blockchain voting. Journal of Cybersecurity **7**(1) (2021)
22. Pedersen, T.P.: A threshold cryptosystem without a trusted party. In: Advances in Cryptology – EUROCRYPT '91. LNCS, vol. 547, pp. 522–526. Springer (1991)
23. People's union for civil liberties v. union of india. Supreme Court of India, (2013) 10 SCC 1 (2013)
24. Justice K.S. puttaswamy (retd.) v. union of india. Supreme Court of India, (2017) 10 SCC 1; and (2019) 1 SCC 1 (2017)
25. The representation of the people act, 1951 (act no. 43 of 1951). Government of India, India Code (1951)
26. Schnorr, C.P.: Efficient signature generation by smart cards. Journal of Cryptology **4**(3), 161–174 (1991)
27. Shanks, D.: Class number, a theory of factorization, and genera. In: 1969 Number Theory Institute. Proceedings of Symposia in Pure Mathematics, vol. 20, pp. 415–440. American Mathematical Society (1971)
28. Springall, D., Finkenauer, T., Durumeric, Z., Kitcat, J., Hursti, H., MacAlpine, M., Halderman, J.A.: Security analysis of the Estonian internet voting system. In: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 703–715. ACM (2014)
29. Terelius, B., Wikström, D.: Proofs of restricted shuffles. In: Progress in Cryptology – AFRICACRYPT 2010. LNCS, vol. 6055, pp. 100–113. Springer (2010)